Linux C++ 服务端开发：OS与网络 1天极限突击指南 (终极版)

针对2年经验的 C++ Linux 后端开发，面试官不仅看重 OS 概念，更看重**底层原理的工程落地**（如线程池设计、锁的选型、排障）。

建议按照以下优先级进行一天的高效复习：

🕑 早上 09:00 - 12:00：进程、线程与并发（绝对高频，占40%）

这是面试的重灾区，必须把概念和实际 C++ 编程（尤其是多线程）结合起来。

1. 进程、线程与上下文切换

- **本质区别**：
  - **进程**：资源分配的基本单位（有独立虚拟地址空间）。
  - **线程**：CPU调度的基本单位（共享进程地址空间）。
  - **协程**：用户态轻量级线程（程序员/运行时调度，无内核切换开销）。
- 🔥 **黄金话术（直接背诵）**：

  > “进程切换比线程切换重得多。因为进程有独立的地址空间，切换时不仅涉及页表切换，还会导致 TLB（快表）失效和 CPU 缓存刷新；而线程共享地址空间，上下文切换主要只涉及寄存器、栈和调度信息的保存与恢复。”

- **进程间通信 (IPC) 最快的方式**：**共享内存**。直接映射到各进程虚拟地址空间，省去内核态到用户态的拷贝，需配合信号量同步。

2. 锁与同步底层机制 (C++工程选型)

不仅要知道有什么锁，还要知道**什么时候用什么**：

- **互斥锁 (Mutex)**：获取不到锁时陷入内核态休眠。**适用**：临界区较长、预计等待时间较长的场景，避免空转耗 CPU。
- **自旋锁 (Spinlock)**：获取不到锁时在用户态 `while` 循环空转。**适用**：多核CPU、临界区极短且锁持有时间可预期的场景。**注意**：单核无意义；若竞争激烈或临界区偏长，会导致 CPU 严重空转浪费。
- **CAS 与 原子变量 (`std::atomic`)**：硬件级别指令，无锁编程基础。**适用**：简单的计数器、状态标志位。
  - **高频追问**：CAS 有什么风险？**答**：会有 **ABA 问题**。复杂的无锁数据结构通常需要配合版本号（Version）、Hazard Pointer 或更谨慎的内存回收机制来规避。
- **读写锁 (RW-Lock)**：**适用**：读多写少的场景。
- **条件变量 (Condition Variable)**：用于线程间等待/通知。
  - **高频追问**：为什么 `wait` 必须配合 `while` 而不是 `if` ？**答**：防止**虚假唤醒**（Spurious Wakeup）。线程被唤醒时不一定条件真的满足，必须再次检查。

3. 死锁 (Deadlock)

- **四个必要条件**：互斥、请求并保持、不可剥夺、循环等待。
- **工程规避方案**：
  - 统一且固定的加锁顺序（破坏循环等待）。
  - 缩小临界区，细化锁粒度。
  - 使用 C++11 的 `std::lock` 同时锁住多个 mutex（防死锁神器），或使用超时机制（`try_lock`）。
- **线上排查思路**：通过 `top -Hp` 找线程，再用 `gdb -p <PID>` 结合 `info threads` 和 `thread apply all bt`，查看多线程调用栈，寻找互相等待的锁（如卡在 `__lll_lock_wait`）。

4. 线程池设计 (C++后端最常考落地题)

把 OS 知识落地到具体代码，面试官最爱考：

- **核心组成**：
  - **任务队列**：存放待处理任务（`std::queue<std::function<void()>>`）。
  - **工作线程集合**：预先创建好的 `std::vector<std::thread>`。
  - **互斥锁（`mutex`）**：保护任务队列的并发访问。
  - **条件变量（`condition_variable`）**：队列为空时阻塞线程，有新任务时唤醒。
  - **停止标志（`stop_flag`）**：控制线程池优雅退出。
- **高频追问**：工作线程为什么在 `while(true)` 里拿任务？**答**：为了让线程不退出、持续复用。循环内部若队列为空，则通过条件变量 `wait` 阻塞。
- **优雅关闭**：设置 `stop_flag = true` -> `cv.notify_all()` 唤醒所有阻塞线程 -> 等待所有线程执行完当前任务后退出（`thread.join()`）。

🕑 中午 13:30 - 15:30：内存管理与文件 I/O（占25%）

1. 虚拟内存、TLB 与缺页中断

- **为什么需要虚拟内存？** 隔离进程、提供连续统一的地址空间。
- **页表与 TLB**：逻辑地址通过**页表**映射到物理地址。**TLB（快表）** 是 CPU 内部的缓存，用来加速地址转换。
- **缺页中断 (Page Fault)**：
  - **Minor (软缺页)**：页已在物理内存中（如共享库），只是当前进程页表未建立映射。
  - **Major (硬缺页)**：数据不在内存中，需要从磁盘调页，**代价极大**。

2. Page Cache (页缓存) 与零拷贝

- **Page Cache 认知框架**：用户读写文件，通常先经过内核的 **Page Cache**。**首次读文件可能需要从磁盘调页进内存（Major Fault），二次读取则直接命中页缓存，所以速度更快。**
- **零拷贝对比**：

- **mmap**：将文件映射到进程虚拟地址空间。**适用**：大文件读写、多进程共享内存映射。
- **sendfile**：数据直接在内核态中从文件描述符传到 Socket，减少用户态参与和数据拷贝。**适用**：静态文件传输（如 Nginx）。

🕑 下午 15:30 - 18:00：网络模型、epoll 与 TCP 基础（占25%）

1. epoll 核心面经 (严谨表述版)

- **为什么比 select/poll 高效？** （⚠️ 别盲目说绝对 O(1)）
  - 核心优势在于：**不需要每次调用都线性扫描和重复拷贝整个 fd 集合**。
  - `epoll_ctl` 维护红黑树（时间复杂度 $O(\log N)$）。
  - 只有活跃的 fd 会通过中断回调放入**就绪双向链表**中，`epoll_wait` 直接返回该链表。所以在**海量并发连接、少量活跃连接**的场景下性能极佳。

2. LT 与 ET 模式

- **LT (水平触发)**：只要读写缓冲区有数据，就会一直通知。
- **ET (边缘触发)**：只有状态发生变化（新数据到达）时才通知一次。
  - **核心陷阱**：使用 ET 模式，**必须使用非阻塞 Socket**，且每次触发必须用 `while` 循环调用 `read`，直到返回 `EAGAIN`。否则会漏读数据导致死锁。

3. TCP 基础与粘包拆包 (epoll 必连问)

- **TIME_WAIT 与 CLOSE_WAIT (🔥 高频对比)**：
  - **TIME_WAIT**：主动关闭方最后停留的状态（等待 2MSL）。**排障**：大量出现说明**本端在频繁主动断开连接**（如短连接未复用）。
  - **CLOSE_WAIT**：被动关闭方收到 FIN 后的状态。**排障**：大量出现通常说明对端已关闭，但**本端应用层代码有 Bug，没有及时调用** `close()` 释放资源。
- **粘包与拆包**：TCP 是面向字节流的，无边界。
  - **常用解决**：固定长度、特殊分隔符、**包头加长度字段 (Length Header)（最常用）**。
- **高并发为何必须 非阻塞 + 事件驱动？**
  - 阻塞 Socket 会导致大量线程挂起等待 I/O。**线程一多，上下文切换和调度开销会急剧上升，直接拖垮 CPU。因此高并发下更常用 epoll + 非阻塞 + 事件驱动**，只在数据就绪后再进行读写。

🚀 附录：考前 10 分钟必背的 6 个黄金口述答案

如果马上就要面试了，请默念这 6 段话：

1. **进程和线程区别？** “进程是资源分配单位，线程是调度单位。进程切换重，涉及页表切换、TLB 失效；线程共享地址空间，切换轻，主要切寄存器和栈。”
2. **Mutex、自旋锁、Atomic 怎么选？** “临界区较长、等待久用 Mutex，避免 CPU 空转；多核且临界区极短用自旋锁；简单的计数器/状态位优先用 Atomic/CAS，但要注意 ABA 问题。”

3. **条件变量为什么要配合 while？** “为了防止虚假唤醒。线程被唤醒时不一定条件真的满足，必须在 while 循环里再次检查条件。”
4. **死锁怎么产生，怎么避免？** “产生是因为多线程互相持有资源并等待对方（四个必要条件）。工程上通常通过：统一加锁顺序、缩小临界区、或者使用 `std::lock` 同时锁住多个 Mutex 来避免。”
5. **epoll 为什么比 select/poll 高效？** “因为 select/poll 每次都要拷贝全量 fd 集合并线性遍历。而 epoll 在内核用红黑树管理 fd，网卡数据到达时通过中断回调将就绪 fd 放入双向链表，`epoll_wait` 直接返回就绪列表，特别适合海量连接且活跃连接较少的场景。”
6. **TIME_WAIT 和 CLOSE_WAIT 区别？** “TIME_WAIT 属于主动关闭方，大量出现说明我们在频繁主动关连接；CLOSE_WAIT 属于被动关闭方，大量出现通常是我们代码有 Bug，对端关了但我们没调用 close()。”